

FAQ Chatbot using Sentence Embeddings

NLP Assignment Report

Course: Natural Language Processing

Assignment: Retrieval-Based FAQ Chatbot

Date: May 30, 2026

Submitted By:

- **Naqcho Ali** - CS-113
 - **Syed Hassham Ahmed** - CS-118
-

Table of Contents

1. [Introduction](#)
 2. [Dataset Description](#)
 3. [Methodology](#)
 4. [Implementation](#)
 5. [Experimental Results](#)
 6. [Error Analysis and Discussion](#)
 7. [Conclusion](#)
 8. [References](#)
-

1. Introduction

1.1 Objective

This project implements a retrieval-based FAQ chatbot designed to answer questions related to university admissions. The chatbot uses semantic similarity matching to identify the most relevant FAQ question from a predefined dataset and returns the corresponding answer.

1.2 Domain Selection

We selected the **University Admissions** domain for this chatbot because:

- It represents a real-world use case with high demand for automated support
- The domain has well-defined question-answer pairs
- Questions often have semantic variations (paraphrasing) which tests the chatbot's understanding

- It provides diverse categories (admissions, fees, academic policies, campus life, etc.)

1.3 Motivation

University admissions offices receive hundreds of repetitive queries daily. An intelligent FAQ chatbot can:

- Provide instant 24/7 responses to common questions
 - Reduce workload on administrative staff
 - Improve user experience for prospective students
 - Handle multiple queries simultaneously
 - Provide consistent and accurate information
-

2. Dataset Description

2.1 Dataset Source

The FAQ dataset was manually created based on common university admissions queries. Questions and answers were formulated to cover comprehensive aspects of the university admission process.

2.2 Dataset Statistics

- **Total FAQ Pairs:** 58
- **Format:** CSV (Comma-Separated Values)
- **Encoding:** UTF-8
- **Columns:** question, answer, category

2.3 Category Distribution

The dataset covers the following categories:

Category	Number of FAQs	Description
Admissions	10	Application process, requirements, deadlines
Programs	6	Available programs, duration, specializations
Fees	6	Tuition, payment plans, financial aid
Academic	13	Grading, attendance, course registration
Contact	7	Office hours, helplines, communication channels
International	8	Visa, scholarships, support for international students

Category	Number of FAQs	Description
Campus Life	8	Facilities, clubs, hostel, transportation

2.4 Sample Entries

Question: "What are the admission requirements for undergraduate programs?"
 Answer: "Applicants must have completed high school with a minimum 60% aggregate.

SAT or equivalent test scores are required for international students."

Category: Admissions

Question: "Is there a Computer Science program?"

Answer: "Yes we offer BS Computer Science with specializations in AI Data Science

and Software Engineering."

Category: Programs

Question: "What support services are available for international students?"
 Answer: "We provide airport pickup orientation programs cultural integration support

and dedicated international student advisor."

Category: International

3. Methodology

3.1 Preprocessing

Text preprocessing is handled by the `TextPreprocessor` class in `preprocessing.py`. The preprocessing pipeline includes:

3.1.1 Light Preprocessing (Used for Embeddings)

For sentence transformer models, we use minimal preprocessing to preserve semantic meaning:

1. **Lowercasing:** Convert all text to lowercase for consistency
2. **Whitespace Normalization:** Remove extra spaces and normalize whitespace
3. **Basic Cleaning:** Remove special characters while preserving sentence structure

Rationale: Sentence transformers are trained on natural language and perform better with minimal preprocessing. Over-aggressive preprocessing can destroy semantic information.

3.1.2 Heavy Preprocessing (Available for TF-IDF)

For traditional methods, more aggressive preprocessing is available:

1. **Tokenization:** Split text into individual words using NLTK
2. **Stopword Removal:** Remove common words (the, is, at, etc.) using NLTK stopwords
3. **Lemmatization:** Reduce words to base form using WordNet lemmatizer
4. **Punctuation Removal:** Strip all punctuation marks

3.2 Text Representation

We use **Sentence Transformers** for text representation, specifically the `all-MiniLM-L6-v2` model.

3.2.1 Why Sentence Transformers?

- **Semantic Understanding:** Captures meaning beyond keyword matching
- **Pre-trained:** Trained on large corpus, no training required
- **Efficient:** Lightweight model (80MB) with fast inference
- **State-of-the-art:** Better performance than TF-IDF for semantic similarity

3.2.2 Model Details

- **Model Name:** `all-MiniLM-L6-v2`
- **Architecture:** Transformer-based (BERT variant)
- **Embedding Dimension:** 384
- **Training:** Trained on 1 billion sentence pairs
- **Performance:** Balances speed and accuracy

3.2.3 Embedding Process

1. Preprocess all FAQ questions using light preprocessing
2. Encode questions into 384-dimensional vectors using the sentence transformer
3. Store embeddings in memory for fast retrieval
4. For user queries, encode using the same model
5. Compare query embedding with all FAQ embeddings

3.3 Similarity Computation

We use **Cosine Similarity** to measure semantic similarity between query and FAQ embeddings.

3.3.1 Cosine Similarity Formula

$$\text{similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

Where:

- A and B are embedding vectors
- $\cdot$ represents dot product
- $||A||$ represents the magnitude (L2 norm) of vector A

3.3.2 Why Cosine Similarity?

- **Scale Invariant:** Measures angle between vectors, not magnitude
- **Range:** Returns values between -1 and 1 (or 0 and 1 for normalized vectors)

- **Interpretable:** Higher values indicate greater similarity
- **Efficient:** Fast computation using vectorized operations

3.3.3 Confidence Threshold

We implement a confidence threshold mechanism:

- **Threshold Value:** 0.5 (configurable)
- **High Confidence:** Similarity > 0.7
- **Medium Confidence:** $0.5 \leq \text{Similarity} \leq 0.7$
- **Low Confidence:** Similarity < 0.5 (fallback response)

When similarity is below the threshold, the chatbot returns:

"Sorry, I could not find a reliable answer to your question. Please try rephrasing or contact our support team."

4. Implementation

4.1 System Architecture

The system consists of four main modules:

```
src/  
├── preprocessing.py    # Text preprocessing utilities  
├── chatbot.py         # Main chatbot logic and interface  
└── evaluation.py      # Evaluation and testing utilities  
  
data/  
├── faqs_clean.csv     # FAQ dataset (58 entries)  
└── faqs.csv           # Original dataset  
  
tests/  
└── test_queries.csv   # Test queries for evaluation
```

4.2 Libraries Used

Library	Version	Purpose
sentence-transformers	2.2.2	Sentence embeddings
scikit-learn	1.3.0	Cosine similarity computation
pandas	2.0.3	Data handling and CSV processing
numpy	1.24.3	Numerical operations
nltk	3.8.1	Text preprocessing (tokenization, lemmatization)

4.3 Key Components

4.3.1 TextPreprocessor Class

```
class TextPreprocessor:
    - preprocess_light(): Minimal preprocessing for embeddings
    - preprocess_heavy(): Aggressive preprocessing for TF-IDF
    - tokenize(): Word tokenization
    - remove_stopwords(): Stopword removal
    - lemmatize(): Word lemmatization
```

4.3.2 FAQChatbot Class

```
class FAQChatbot:
    - __init__(): Load dataset and model, encode FAQs
    - get_response(): Get best matching answer for query
    - get_top_k_responses(): Get top K similar FAQs
    - chat(): Interactive command-line interface
```

4.3.3 ChatbotEvaluator Class

```
class ChatbotEvaluator:
    - evaluate_test_queries(): Batch evaluation on test set
    - evaluate_single_query(): Single query evaluation
    - generate_report(): Generate evaluation report
```

4.4 Workflow

1. Initialization:

- Load FAQ dataset from CSV
- Initialize sentence transformer model
- Preprocess and encode all FAQ questions
- Store embeddings in memory

2. Query Processing:

- Accept user query
- Preprocess query using light preprocessing
- Encode query into embedding vector
- Compute cosine similarity with all FAQ embeddings
- Identify best match

3. Response Generation:

- Check if similarity exceeds confidence threshold
- If yes: Return matched answer with confidence level
- If no: Return fallback response

4. Interactive Chat:

- Display welcome message

- Accept user input in loop
 - Process query and display response
 - Show similarity score and matched question
 - Continue until user types 'quit' or 'exit'
-

5. Experimental Results

5.1 Test Query Design

We designed 20 test queries covering various scenarios:

1. **Exact Matches:** Queries identical to FAQ questions
2. **Paraphrased Queries:** Semantically similar but differently worded
3. **Incomplete Queries:** Partial or abbreviated questions
4. **Spelling Errors:** Queries with typos
5. **Ambiguous Queries:** Questions that could match multiple FAQs
6. **Out-of-Domain:** Questions not covered in the dataset

5.2 Sample Results

5.2.1 Successful Examples

Query 1: "How do I apply?"

Matched FAQ: "How can I apply for admission?"

Answer: "You can apply online through our university portal. Fill out the application form and upload required documents."

Similarity Score: 0.782

Confidence: HIGH

Analysis: Successfully matched despite abbreviated query.

Query 2: "What's the minimum percentage needed?"

Matched FAQ: "What is the minimum GPA requirement?"

Answer: "The minimum GPA requirement is 2.5 on a 4.0 scale or 60% aggregate marks."

Similarity Score: 0.691

Confidence: MEDIUM

Analysis: Correctly understood "percentage" relates to GPA/admission requirements.

Query 3: "Do you have CS program?"

Matched FAQ: "Is there a Computer Science program?"

Answer: "Yes we offer BS Computer Science with specializations in AI Data Science and Software Engineering."

Similarity Score: 0.845

Confidence: HIGH

Analysis: Handled abbreviation "CS" and informal phrasing well.

Query 4: "Can international students get scholarships?"

Matched FAQ: "Are there scholarships for international students?"

Answer: "Yes we offer limited merit-based scholarships for international students with exceptional academic records."

Similarity Score: 0.891

Confidence: HIGH

Analysis: Perfect semantic match with paraphrased query.

Query 5: "What if I don't pass a subject?"

Matched FAQ: "What happens if I fail a course?"

Answer: "Failed courses must be repeated. You can retake the course in the next semester when it is offered."

Similarity Score: 0.734

Confidence: HIGH

Analysis: Understood "don't pass" means "fail" in academic context.

5.2.2 Moderate Performance

Query 6: "How much does it cost?"

Matched FAQ: "What is the fee structure?"

Answer: "Fee structure varies by program. Undergraduate programs range from PKR 150000 to PKR 250000 per semester."

Similarity Score: 0.623

Confidence: MEDIUM

Analysis: Correctly identified cost-related query but similarity is moderate due to vague phrasing.

Query 7: "When can I start?"

Matched FAQ: "When does the academic year start?"

Answer: "The academic year consists of Fall (September) and Spring (February) semesters."

Similarity Score: 0.598

Confidence: MEDIUM

Analysis: Ambiguous query - could refer to admission process or academic year. Matched academic year.

5.2.3 Failure Cases

Query 8: "What about parking?"

Matched FAQ: "Is transportation provided?"

Answer: "Yes university buses operate on multiple routes. Transport fee is separate and optional."

Similarity Score: 0.512

Confidence: MEDIUM

Analysis: Parking information not in dataset. Matched closest transportation-related FAQ but answer is not relevant.

Query 9: "Can I get a refund?"

Matched FAQ: "Is there an application fee?"

Answer: "Yes there is a non-refundable application fee of PKR 2000 for local students and \$50 for international students."

Similarity Score: 0.487

Confidence: LOW (Fallback)

Analysis: Refund policy not in dataset. Similarity below threshold, fallback response triggered correctly.

Query 10: "What's the weather like?"

Matched FAQ: "What are the office hours?"

Answer: "Office hours are Monday to Friday 9 AM to 5 PM. Offices are closed on weekends and public holidays."

Similarity Score: 0.312

Confidence: LOW (Fallback)

Analysis: Completely out-of-domain query. Low similarity correctly triggers fallback.

5.3 Quantitative Results

Based on 20 test queries:

Metric	Value
Correct Matches	15/20 (75%)
Partially Correct	3/20 (15%)
Incorrect Matches	2/20 (10%)
Average Similarity (Correct)	0.742
Average Similarity (Incorrect)	0.421
Fallback Triggered	4/20 (20%)

5.4 Performance by Query Type

Query Type	Success Rate
Exact Match	100% (3/3)
Paraphrased	85% (6/7)
Incomplete	70% (3/4)
With Typos	60% (2/3)
Out-of-Domain	0% (0/3) - Correctly rejected

6. Error Analysis and Discussion

6.1 Semantic Limitations

6.1.1 Ambiguous Queries

Issue: Queries like "When can I start?" are ambiguous - they could refer to:

- When admission process starts
- When academic year starts
- When course registration starts

Impact: The chatbot matches based on highest similarity but may not capture user intent.

Solution: Implement clarification questions or context tracking in future versions.

6.1.2 Implicit Information

Issue: Queries like "How much?" lack explicit context about what the user is asking about (fees, application cost, hostel charges).

Impact: May match incorrect FAQ or return low confidence.

Solution: Encourage users to be more specific or implement multi-turn conversations.

6.2 Dataset Quality Issues

6.2.1 Coverage Gaps

Issue: Dataset doesn't cover all possible questions (e.g., parking, refund policies, alumni services).

Impact: Out-of-scope queries either match irrelevant FAQs or trigger fallback.

Solution: Continuously expand dataset based on user queries and feedback.

6.2.2 Answer Completeness

Issue: Some answers could be more detailed or provide additional resources.

Impact: Users may need follow-up questions.

Solution: Enhance answers with links, contact information, or related FAQs.

6.3 Paraphrasing Challenges

6.3.1 Synonyms and Variations

Strength: Sentence transformers handle synonyms well:

- "cost" ↔ "fee" ↔ "charges"
- "fail" ↔ "don't pass"
- "CS" ↔ "Computer Science"

Limitation: Domain-specific jargon or abbreviations not in training data may not be recognized.

6.3.2 Sentence Structure

Strength: Model handles different sentence structures:

- "What are the requirements?" ↔ "Requirements for admission?"
- "Can I apply?" ↔ "How to apply?"

Limitation: Very long or complex sentences may dilute semantic signal.

6.4 Spelling Error Handling

6.4.1 Minor Typos

Performance: Model shows some robustness to minor typos:

- "addmission" → "admission" (still matches)
- "scholership" → "scholarship" (still matches)

Reason: Subword tokenization in transformers provides partial robustness.

6.4.2 Major Errors

Limitation: Multiple typos or completely misspelled words significantly reduce similarity:

- "whn cn I strt?" → Low similarity, may fail

Solution: Implement spell-checking preprocessing layer.

6.5 Computational Constraints

6.5.1 Model Size vs Performance Trade-off

Current Model: all-MiniLM-L6-v2 (80MB, 384 dimensions)

- **Pros:** Fast inference, low memory, good accuracy
- **Cons:** Less powerful than larger models

Alternatives:

- Larger models (e.g., all-mpnet-base-v2) offer better accuracy but slower
- Smaller models (e.g., all-MiniLM-L12-v2) are faster but less accurate

Decision: Current model provides optimal balance for FAQ chatbot use case.

6.5.2 Scalability

Current Performance:

- 58 FAQs: < 100ms response time
- Encoding time: ~1 second for initialization

Scalability Analysis:

- Linear growth with dataset size
- For 1000+ FAQs, consider indexing (FAISS, Annoy)

- **Current approach** suitable for up to 500-1000 FAQs

6.6 Confidence Threshold Tuning

6.6.1 Current Threshold: 0.5

Analysis:

- **Too Low:** May return irrelevant answers
- **Too High:** May reject valid queries
- **Current (0.5):** Balanced approach

6.6.2 Threshold Impact

Threshold	Precision	Recall	User Experience
0.3	Low	High	Many incorrect answers
0.5	Medium	Medium	Balanced (current)
0.7	High	Low	Many fallbacks

Recommendation: Keep 0.5 for general use, allow user configuration.

6.7 Limitations Summary

1. **No Context Memory:** Each query is independent, no conversation history
2. **No Multi-turn Dialogue:** Cannot ask clarifying questions
3. **Static Dataset:** Requires manual updates, no learning from interactions
4. **English Only:** No multilingual support
5. **No Entity Recognition:** Cannot extract specific entities (dates, names, numbers)
6. **No Personalization:** Same responses for all users

6.8 Strengths Summary

1. **Semantic Understanding:** Goes beyond keyword matching
 2. **Paraphrase Handling:** Robust to different phrasings
 3. **Fast Response:** Real-time performance
 4. **No Training Required:** Uses pre-trained models
 5. **Confidence Mechanism:** Prevents incorrect answers
 6. **Extensible:** Easy to add new FAQs
-

7. Conclusion

7.1 Achievements

We successfully developed a retrieval-based FAQ chatbot with the following accomplishments:

1. **Comprehensive Dataset:** Created 58 FAQ pairs covering 7 categories of university admissions
2. **Semantic Matching:** Implemented sentence transformer-based similarity matching
3. **Robust Preprocessing:** Developed flexible preprocessing pipeline
4. **Confidence Mechanism:** Implemented threshold-based fallback system
5. **Evaluation Framework:** Created systematic evaluation with 20 test queries
6. **High Accuracy:** Achieved 75% correct match rate on diverse test queries
7. **User-Friendly Interface:** Built interactive command-line chatbot

7.2 Lessons Learned

7.2.1 Technical Insights

- **Embeddings > Keywords:** Sentence transformers significantly outperform TF-IDF for semantic similarity
- **Preprocessing Trade-offs:** Minimal preprocessing works better for transformer models
- **Threshold Tuning:** Confidence threshold is critical for user experience
- **Dataset Quality:** High-quality, comprehensive dataset is more important than complex algorithms

7.2.2 Practical Insights

- **User Query Patterns:** Users phrase questions in many different ways
- **Ambiguity is Common:** Many queries lack sufficient context
- **Coverage Matters:** Dataset gaps are the primary source of failures
- **Feedback Loop:** Continuous dataset improvement based on user queries is essential

7.3 Future Improvements

7.3.1 Short-term Enhancements

1. **Spell Checking:** Add preprocessing layer for typo correction
2. **Query Expansion:** Expand user queries with synonyms
3. **Top-K Results:** Show multiple relevant FAQs instead of just one
4. **Category Filtering:** Allow users to filter by category
5. **Feedback Collection:** Add thumbs up/down for answer quality

7.3.2 Medium-term Enhancements

1. **Context Tracking:** Maintain conversation history for follow-up questions
2. **Clarification Questions:** Ask for more details when query is ambiguous
3. **Hybrid Approach:** Combine embeddings with keyword matching
4. **Answer Ranking:** Use learning-to-rank for better result ordering
5. **Analytics Dashboard:** Track common queries and failure patterns

7.3.3 Long-term Vision

1. **Generative Responses:** Integrate GPT-based models for dynamic answer generation
2. **Multilingual Support:** Extend to multiple languages
3. **Voice Interface:** Add speech-to-text and text-to-speech
4. **Personalization:** Adapt responses based on user profile
5. **Active Learning:** Automatically improve from user interactions
6. **Integration:** Connect with university systems for real-time data

7.4 Real-world Deployment Considerations

For production deployment, the following would be needed:

1. **Web Interface:** Build user-friendly web or mobile interface
2. **API Development:** Create REST API for integration
3. **Database Backend:** Store FAQs in database for easy updates
4. **Logging & Monitoring:** Track usage, errors, and performance
5. **Security:** Implement authentication and rate limiting
6. **Scalability:** Use caching and load balancing
7. **A/B Testing:** Test different models and thresholds
8. **Human Handoff:** Escalate complex queries to human agents

7.5 Final Remarks

This project demonstrates the effectiveness of modern NLP techniques for building practical conversational systems. While the chatbot has limitations, it successfully handles the majority of common queries and provides a solid foundation for future enhancements.

The combination of sentence transformers and cosine similarity proves to be a powerful yet simple approach for FAQ retrieval. The key to success lies not in algorithmic complexity, but in dataset quality, thoughtful preprocessing, and careful threshold tuning.

This chatbot can serve as a valuable tool for university admissions offices, reducing workload and improving student experience. With continuous improvement and user feedback, it can evolve into a comprehensive virtual assistant.

8. References

8.1 Libraries and Frameworks

- [1] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. <https://arxiv.org/abs/1908.10084>
- [2] Sentence Transformers Documentation. <https://www.sbert.net/>
- [3] Scikit-learn: Machine Learning in Python. Pedregosa et al., *JMLR* 12, pp. 2825-2830, 2011. <https://scikit-learn.org/>
- [4] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. <https://www.nltk.org/>
- [5] Pandas: Powerful data structures for data analysis. McKinney, W. (2010). <https://pandas.pydata.org/>
- [6] NumPy: The fundamental package for scientific computing with Python. Harris, C.R., et al. (2020). *Nature*, 585, 357–362. <https://numpy.org/>

8.2 Research Papers

- [7] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
- [8] Vaswani, A., et al. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems*, 30.
- [9] Mikolov, T., et al. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*.

8.3 Tutorials and Documentation

- [10] Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers/>
- [11] Python Documentation. <https://docs.python.org/3/>
- [12] Git Documentation. <https://git-scm.com/doc>

8.4 Datasets and Resources

- [13] FAQ Dataset: Manually created for this project based on university admissions information.
- [14] NLTK Corpora: WordNet, Stopwords. https://www.nltk.org/nltk_data/

8.5 Tools and Platforms

- [15] GitHub: Version Control and Collaboration. <https://github.com/>
- [16] Visual Studio Code: Code Editor. <https://code.visualstudio.com/>
- [17] Python Package Index (PyPI). <https://pypi.org/>

End of Report